

Week_8

```
srikan@DESKTOP-0NIMS2G: ~/OSSP
3]+ Stopped ./server
srikan@DESKTOP-0NIMS2G:~/OSSP$ ./client
Enter message: hello
Server: Server received: hello
Enter message: hihihihihihihihhi
Server: Server received: hihihihihihihihhi
Enter message: ^Z
4]+ Stopped
srikan@DESKTOP-0NIMS2G:~/OSSP$ ./client
srikan@DESKTOP-0NIMS2G:~/OSSP$ gedit week_7.c
srikan@DESKTOP-0NIMS2G:~/OSSP$ gcc week_7.c -o week_7
srikan@DESKTOP-0NIMS2G:~/OSSP$ ./week_7
==== Process Memory Addresses ====
Code address : 0x6303bec08209
Global address : 0x6303bec0b010
Static address : 0x6303bec0b014
Heap address : 0x6303d1ab02a0
Stack address : 0x7f7fea23892fc

Process ID (PID): 743

Process is running...
Use another terminal to check /proc/743/maps
Press Enter to exit...

srikan@DESKTOP-0NIMS2G:~/OSSP$
srikan@DESKTOP-0NIMS2G:~/OSSP$ gedit week_7.c
srikan@DESKTOP-0NIMS2G:~/OSSP$ gedit memory.c
srikan@DESKTOP-0NIMS2G:~/OSSP$ gcc memory.c -o memory
srikan@DESKTOP-0NIMS2G:~/OSSP$ ./memory
Memory allocated using malloc():
1 2 3 4 5

Memory allocated using calloc():
0 0 0 0 0

After realloc(), array contains:
1 2 3 4 5 6 7 8 9 10

Memory released using free().
srikan@DESKTOP-0NIMS2G:~/OSSP$ gedit memory.c
```

```
memory.c
14
15 for (i = 0; i < 5; i++) {
16     a[i] = i + 1;
17 }
18
19 printf("Memory allocated using malloc():\n");
20 for (i = 0; i < 5; i++) {
21     printf("%d ", a[i]);
22 }
23 printf("\n");
24
25 // calloc()
26 int *b = (int *)calloc(5, sizeof(int));
27
28 if (b == NULL) {
29     printf("calloc failed\n");
30     free(a);
31     return 1;
32 }
33
34 printf("\nMemory allocated using calloc():\n");
35 for (i = 0; i < 5; i++) {
36     printf("%d ", b[i]);
37 }
38 printf("\n");
39
40 // realloc()
41 a = (int *)realloc(a, 10 * sizeof(int));
42
43 if (a == NULL) {
44     printf("realloc failed\n");
45     free(b);
46     return 1;
47 }
48
49 for (i = 5; i < 10; i++) {
50     a[i] = i + 1;
51 }
52
53 printf("\nAfter realloc(), array contains:\n");
54 for (i = 0; i < 10; i++) {
55     printf("%d ", a[i]);
56 }
57 printf("\n");
58
59 // free()
60 free(a);
61 free(b);
62
63 printf("\nMemory released using free():\n");
64
65 return 0;
66
```

```
srikan@DESKTOP-0NIMS2G: ~/OSSP
srikan@DESKTOP-0NIMS2G:~/OSSP$ valgrind --leak-check=full ./memory
==2063== Memcheck, a memory error detector
==2063== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2063== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2063== Command: ./memory
==2063==
Memory allocated using malloc():
1 2 3 4 5

Memory allocated using calloc():
0 0 0 0 0

After realloc(), array contains:
1 2 3 4 5 6 7 8 9 10

Memory released using free().
==2063==
==2063== HEAP SUMMARY:
==2063==    in use at exit: 0 bytes in 0 blocks
==2063==    total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==2063==
==2063== All heap blocks were freed -- no leaks are possible
==2063==
==2063== For lists of detected and suppressed errors, rerun with: -s
==2063== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
srikan@DESKTOP-0NIMS2G:~/OSSP$ gedit cow.c
```